

أمثلة واقعية كاملة - Async/Await + Try/Catch

race مع إدارة الأخطاء، بحيث تتجنب Node.js في **بأمان وفعالية async كيفية كتابة كود** هذا الملف يوضح conditions، وتتأكد من التعامل مع النتائج بشكل صحيح.

1 async داخل دالة await استخدام

```
async function getUsers() {
  try {
    const result = await db.query("SELECT * FROM users");
    return result.rows; // ربط المنطق مباشرة بالنتيجة
  } catch (err) {
    console.error("DB query failed", err);
    throw err; // إعادة الخطأ للطبقات الأعلى إذا لزم الأمر
  }
}

// استخدام الدالة
getUsers()
  .then(users => console.log("Users fetched:", users))
  .catch(err => console.log("Error caught in main flow:", err));
```

Promise الأخطاء يتم التعامل معها - يمكن استخدام النتيجة مباشرة بعد - race condition مزايًا: لا ❑

2 async/await في Express route استخدام

```
app.get('/users', async (req, res) => {
  try {
    const result = await db.query("SELECT * FROM users");
    res.json(result.rows);
  } catch (err) {
    console.error("DB query failed", err);
    res.status(500).json({ error: "Internal Server Error" });
  }
});
```

بين إرسال الاستجابة وقراءة race condition مزايًا: - المخرجات دائمًا دقيقة - الأخطاء يتم التعامل معها فورًا - لا يوجد ❑ البيانات

3 async عند عدة عمليات race conditions تجنب

```
async function getDashboardData() {
  try {
    const [users, orders, products] = await Promise.all([
      db.query("SELECT * FROM users"),
      db.query("SELECT * FROM orders"),
      db.query("SELECT * FROM products")
    ]);

    return { users: users.rows, orders: orders.rows, products: products.rows };
  } catch (err) {
    console.error("Failed fetching dashboard data", err);
    throw err;
  }
}

getDashboardData()
  .then(data => console.log("Dashboard data:", data))
  .catch(err => console.log("Error:", err));
```

✓ إدارة أخطاء مركزة - جميع النتائج متاحة قبل المتابعة - I/O مزايا: - تنفيذ متوازي لعمليات

4 async داخل Microtask / Macrotask التعامل مع

```
async function complexFlow() {
  console.log("Start");

  setTimeout(() => console.log("Macrotask"), 0);

  await Promise.resolve(); // Microtask
  console.log("After await / Microtask");

  console.log("End of function");
}

complexFlow();
```

✓ بوعي كامل مع await يمكن دمج - timing بال async bugs مزايا: - فهم ترتيب التنفيذ - لا تتأثر

♦ الخلاصة

- `async` لا تعتمد على متغيرات خارجية قبل اكتمال: **ربط المنطق مباشرة بالنتيجة**.
- `try/catch` أو `catch()`. أو `try/catch`.
- فقط `async` داخل دالة `await`.
- للتنفيذ المتوازي الآمن `Promise.all` استخدم.
- `await` و `Promise` و `setTimeout` مهم عند دمج `Microtask` / `Macrotask` فهم ترتيب.

يمكنك استخدامه كأساس لأي مشروع `Node.js` في `async code` هذا نموذج عملي جاهز لجميع سيناريوهات  حقيقي.